# 🏠 Aplikasi Manajemen Kost

Aplikasi web full-stack untuk mengelola sistem kost-kostan, termasuk manajemen kamar, penyewa, booking requests, dan pembayaran dengan sistem real-time.

## 📋 Daftar Isi



---

## ✨ Fitur Utama

### 👨‍💼 Untuk Admin/Pemilik Kost

- **Dashboard Admin**: Overview lengkap dengan statistik real-time
- **Manajemen Properti**: CRUD properti kost dengan multiple kost support
- **Manajemen Kamar**:
  - Tambah, edit, dan hapus kamar
  - Atur harga sewa dan fasilitas
  - Update status ketersediaan kamar
  - Automatic tenant removal saat kamar dikosongkan
- **Manajemen Penyewa**:
  - Lihat daftar penyewa aktif per properti
  - Auto-remove penyewa saat kamar tersedia
- **Booking Management**:
  - Approve/reject permintaan booking
  - Process booking dengan status tracking
  - Automatic room availability update
- **Payment Management**:
  - Setup metode pembayaran (Rekening Bank / QRIS)
  - History pembayaran dengan filter per properti
  - Upload QRIS image ke Supabase Storage
- **Referral System**: Generate kode referral untuk user registration

### 👥 Untuk User/Penyewa

- **Browse Kamar**: Lihat daftar kamar tersedia dengan filter per kost
- **Detail Kamar**: Informasi lengkap harga dan fasilitas
- **Booking System**:
  - Submit booking request dengan durasi custom
  - Real-time notification untuk booking status
  - Countdown timer untuk pembayaran (10 menit)
- **Payment System**:
  - Multi-step payment flow dengan progress indicator
  - Upload bukti pembayaran
  - Tracking status pembayaran
- **Dashboard Penyewa**:
  - Informasi sewa aktif
  - Status pembayaran dan jatuh tempo
  - Alert untuk pembayaran mendekati/lewat jatuh tempo
- **Referral Verification**: Input kode referral dari pemilik kost

---

# 🛠️ Teknologi

## Frontend

- **Framework**: Next.js 14 (App Router)
- **UI Library**: React 18
- **Styling**: Tailwind CSS
- **Icons**: Lucide React
- **State Management**: React Hooks (useState, useEffect, useActionState)
- **Form Handling**: React Server Actions

## Backend

- **Database**: Supabase (PostgreSQL)
- **Authentication**: Supabase Auth (Google OAuth)
- **Storage**: Supabase Storage (payment proofs, QRIS images)
- **Real-time**: Supabase Realtime (booking updates)
- **API**: Next.js API Routes (Route Handlers)

## Development Tools

- **TypeScript**: Type safety
- **ESLint**: Code linting
- **Date-fns**: Date manipulation
- **UUID**: Unique ID generation

---

# 📁 Struktur Aplikasi

```
src/app/
├── api/                                    # API Routes
│   ├── check-expired-bookings/             # Cron job untuk expired bookings
│   ├── get-kosts/                          # Get kosts by referral
│   ├── kosts/                              # CRUD kosts
│   ├── rooms/                              # CRUD rooms
│   ├── validate-referral/                  # Validate referral code
│   └── kosts/clear-tenant/                 # Clear tenants from room
│
├── auth/callback/                          # OAuth Callbacks
│   ├── admin/                              # Admin callback
│   └── user/                               # User callback
│
├── dashboard-admin/                        # Admin Dashboard
│   ├── layout.tsx                          # Admin layout dengan sidebar
│   ├── page.tsx                            # Dashboard summary
│   ├── booking-request/                    # Booking management
│   │   ├── page.tsx
│   │   ├── actions.ts
│   │   └── components/
│   │       └── SubmitButton.tsx
│   ├── manage-rooms/                       # Room management
│   │   ├── page.tsx                        # List rooms
│   │   ├── actions.ts
│   │   ├── [kostId]/manage/[roomId]/     # Edit room
│   │   ├── add/                            # Add room
│   │   ├── add-property/                   # Add kost
│   │   └── components/
│   │       ├── DeleteRoomButton.tsx
│   │       └── RoomAvailabilityCheckbox.tsx
│   ├── payment-history/                    # Payment history
│   │   └── page.tsx
│   └── payment-method/                     # Payment methods setup
│       ├── page.tsx
│       ├── actions.ts
│       └── components/
│           └── PaymentMethodForm.tsx
│
├── dashboard-penyewa/                      # Tenant Dashboard
│   ├── page.tsx                            # Tenant info & due date
│   └── LogoutButton.tsx
│
├── dashboard-user/                         # User Dashboard
│   ├── page.tsx                            # Browse rooms
│   ├── layout.tsx
│   ├── actions.tsx                         # Booking actions
│   ├── payments/actions.ts                 # Payment actions
│   └── components/
│       ├── BookingForm.tsx
│       ├── BookingModal.tsx
│       ├── CountdownTimer.tsx
```

```
│       ├── Header.tsx
│       ├── NotificationModal.tsx
│       ├── PaymentModal.tsx
│       ├── RoomCard.tsx
│       ├── RoomList.tsx
│       └── SubmitForm.tsx
│
├── login/                                  # Login Pages
│   ├── admin/page.tsx                      # Admin login (Google OAuth)
│   └── user/page.tsx                       # User login (Google OAuth)
│
├── referral-input/                         # Referral code input
│   └── page.tsx
│
├── coming-soon/                            # Coming soon page
│   └── page.tsx
│
└── page.tsx                                # Landing/Role selection page
```

---

# 📦 Instalasi

## Prerequisites

- Node.js 18.x atau lebih tinggi
- npm/yarn/pnpm/bun
- Akun Supabase

## Langkah Instalasi

1. **Clone repository**

```
git clone <repository-url>
cd <project-folder>
```

2. **Install dependencies**

```
npm install
# atau
yarn install
# atau
pnpm install
```

3. **Setup Environment Variables**

Buat file `.env` di root project:

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=your_supabase_project_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
SUPABASE_SERVICE_ROLE_KEY=your_supabase_service_role_key
```

```
# App URL
NEXT_PUBLIC_SITE_URL=https://web-kos.vercel.app
```

4. **Setup Database**

Jalankan SQL migration di Supabase SQL Editor:

```
-- Lihat bagian "Struktur Database" di bawah untuk schema lengkap
```

5. **Setup Supabase Storage Buckets**

Buat 2 buckets di Supabase Storage:

- `payment_file` (untuk bukti pembayaran)
- `payment_methods` (untuk QRIS images)

Set kebijakan public access untuk kedua buckets.

6. **Setup Google OAuth**

- Tambahkan Google sebagai OAuth provider di Supabase Auth
- Set redirect URLs:
  - `http://localhost:3000/auth/callback/admin`
  - `http://localhost:3000/auth/callback/user`

7. **Run Development Server**

```
npm run dev
# atau
yarn dev
```

Aplikasi akan berjalan di `http://localhost:3000`

---

# 🗄️ Struktur Database

## Tables

### 1. `profiles`

User profile dengan role-based system

```sql
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id),
  email TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('pemilik', 'penyewa')),
  referral_code TEXT UNIQUE,
  used_referral_code TEXT,
  referral_verified BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

**2. `kosts`**

Properti kost yang dimiliki pemilik

```
CREATE TABLE kosts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  owner_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  address TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);
```

**3. `rooms`**

Kamar-kamar di setiap kost

```
CREATE TABLE rooms (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  kost_id UUID REFERENCES kosts(id) ON DELETE CASCADE,
  room_number TEXT NOT NULL,
  price NUMERIC NOT NULL,
  facilities TEXT,
  is_available BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

**4. `booking_requests`**

Permintaan booking dari user

```
CREATE TABLE booking_requests (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  room_id UUID REFERENCES rooms(id) ON DELETE CASCADE,
  kost_id UUID REFERENCES kosts(id) ON DELETE CASCADE,
  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'process',
         'approved', 'rejected', 'expired')),
  due_date DATE NOT NULL,
  booking_expires_at TIMESTAMP,
  paid_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

**5. `penyewa`**

Data penyewa aktif yang sudah approved

```
CREATE TABLE penyewa (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  penyewa_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  room_id UUID REFERENCES rooms(id) ON DELETE CASCADE,
```

```
  kost_id UUID REFERENCES kosts(id) ON DELETE CASCADE,
  start_date DATE DEFAULT CURRENT_DATE,
  end_date DATE,
  due_date DATE NOT NULL,
  status TEXT DEFAULT 'active',
  created_at TIMESTAMP DEFAULT NOW()
);
```

#### 6. `payments`

History pembayaran

```
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  penyewa_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  room_id UUID REFERENCES rooms(id) ON DELETE CASCADE,
  kost_id UUID REFERENCES kosts(id) ON DELETE CASCADE,
  booking_request_id UUID REFERENCES booking_requests(id),
  amount NUMERIC NOT NULL,
  due_date DATE NOT NULL,
  proof_url TEXT,
  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'approved',
         'rejected', 'paid', 'overdue')),
  created_at TIMESTAMP DEFAULT NOW()
);
```

#### 7. `payment_methods`

Metode pembayaran yang disediakan pemilik

```
CREATE TABLE payment_methods (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  kost_id UUID REFERENCES kosts(id) ON DELETE CASCADE,
  type TEXT NOT NULL CHECK (type IN ('rekening', 'qris')),
  bank_name TEXT,
  account_number TEXT,
  qris_url TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

## Database Relationships

```
profiles (pemilik)
  └── kosts (1:N)
        ├── rooms (1:N)
        │     ├── booking_requests (1:N)
        │     ├── penyewa (1:N)
        │     └── payments (1:N)
        └── payment_methods (1:N)

profiles (penyewa)
```

```
├── booking_requests (1:N)
├── penyewa (1:N)
└── payments (1:N)
```

### Cascade Delete Order

Saat menghapus data, urutan penting untuk menghindari foreign key constraint errors:

1. **Delete Room**:
   - booking_requests → payments → penyewa → rooms
2. **Clear Tenant**:
   - penyewa → payments → booking_requests
3. **Delete Kost**:
   - Semua dependencies terhapus otomatis via CASCADE

---

# 🔌 API Endpoints

## 1. `/api/check-expired-bookings` (GET/POST)

**Purpose**: Cron job untuk menghapus booking yang expired (>10 menit tanpa pembayaran)

**Authentication**: Bearer token dengan `CRON_SECRET` (POST only)

**Logic**:

- Query booking_requests dengan status 'pending', `booking_expires_at` < now, dan `paid_at` NULL
- Update status booking menjadi 'expired'
- Set room kembali `is_available: true`

**Response**:

```
{
  "success": true,
  "message": "X expired booking(s) processed",
  "count": 2,
  "bookings": [...]
}
```

---

## 2. `/api/get-kosts` (GET)

**Purpose**: Get semua kost milik owner berdasarkan referral code user

**Authentication**: Supabase Auth (Required)

**Logic**:

- Cek user terautentikasi
- Ambil profile user untuk mendapatkan `used_referral_code`
- Cari owner dengan `referral_code` yang sesuai
- Return semua kost milik owner tersebut

**Response**:

```
{
  "success": true,
  "owner": {
    "id": "uuid",
    "email": "owner@example.com"
  },
  "kosts": [
    {
      "id": "uuid",
      "name": "Kost Melati",
      "address": "Jl. Mawar No. 123"
    }
  ]
}
```

**Error Responses**:

- `401`: Unauthorized (belum login)
- `403`: Referral not verified
- `404`: Owner not found

---

## 3. `/api/kosts` (GET/POST)

**Purpose**: CRUD kost properties

**GET - List Kosts**

- Authentication: Required
- Return: Semua kost milik owner yang login

**POST - Create Kost**

- Authentication: Required
- Body:

```
{
  "owner_id": "uuid",
  "name": "Kost Melati",
  "address": "Jl. Mawar No. 123"
}
```

- Validation: name dan address tidak boleh kosong

- Return: Data kost yang baru dibuat

---

## 4. `/api/rooms` (GET/POST)

**Purpose**: CRUD rooms

**GET - List Rooms**

- Authentication: Required
- Return: Semua rooms di database

**POST - Create Room**

- Authentication: Required
- Body:

```
{
  "kost_id": "uuid",
  "room_number": "A101",
  "price": 1500000,
  "facilities": "WiFi, AC, Kamar Mandi Dalam",
  "is_available": true
}
```

---

## 5. `/api/kosts/clear-tenant` (POST)

**Purpose**: Clear semua penyewa dari room dan reset availability

**Authentication**: Required

**Body**:

```
{
  "roomId": "uuid"
}
```

**Logic**:

1. Hapus semua penyewa dari room
2. Set room `is_available: true`
3. Hapus semua booking_requests untuk room
4. Update payments terkait menjadi ‘cancelled’

---

## 6. `/api/validate-referral` (POST)

**Purpose**: Validasi kode referral dari owner

**Authentication**: Required

**Body**:

```
{
  "referralCode": "QWERTY"
}
```

**Logic**:

- Cari owner dengan `referral_code` sesuai (case insensitive)
- Cek role owner harus 'pemilik'
- Update user profile: set `used_referral_code` dan `referral_verified: true`
- Return info owner dan jumlah kost

**Response**:

```
{
  "success": true,
  "message": "Kode referral berhasil diverifikasi",
  "owner": {
    "id": "uuid",
    "email": "owner@example.com"
  },
  "kostsCount": 3
}
```

**Debug Mode**: Return available referral codes jika tidak ditemukan (untuk development)

---

# 🔄 Workflow Aplikasi

## 🔐 Authentication Flow

### Admin/Pemilik Registration & Login

```
1. Navigate ke /login/admin
2. Click "Login with Google"
   ↓
3. Google OAuth (Supabase Auth)
   ↓
4. Redirect ke /auth/callback/admin
   ↓
5. Check existing profile:
   - Jika belum ada: Create profile dengan role 'pemilik'
   - Jika sudah ada dengan role berbeda: Error & logout
   - Jika sudah 'pemilik': Pass
   ↓
6. Redirect ke /dashboard-admin
```

**Error Handling**:

- `already-registered`: User sudah terdaftar sebagai penyewa
- `profile-creation-failed`: Gagal create profile (tampilkan detail error)
- `oauth-failed`: OAuth gagal

---

### User/Penyewa Registration & Login

```
1. Navigate ke /login/user
2. Click "Login with Google"
   ↓
3. Google OAuth (Supabase Auth)
   ↓
4. Redirect ke /auth/callback/user
   ↓
5. Check profile.referral_verified:
   - FALSE: Redirect ke /referral-input
   - TRUE: Redirect ke /dashboard-user
```

### Referral Input Flow:

```
/referral-input
   ↓
1. User input kode referral
2. POST /api/validate-referral
   ↓
3. If valid:
   - Update profile.used_referral_code
   - Set referral_verified = TRUE
   - Redirect ke /dashboard-user
   ↓
4. If invalid:
   - Show error modal
   - User bisa coba lagi
```

---

## 🏠 Property & Room Management Flow

### Tambah Properti Baru

```
/dashboard-admin/manage-rooms/add-property
   ↓
1. Fill form:
   - Nama properti
   - Alamat lengkap
   ↓
2. Submit → POST /api/kosts
   ↓
3. Redirect ke /dashboard-admin/manage-rooms?kost_id={newKostId}
```

---

### Tambah Kamar Baru

```
/dashboard-admin/manage-rooms/add?kost_id={kostId}
   ↓
1. Fill RoomForm:
   - Nomor kamar
   - Harga sewa
   - Fasilitas (predefined + custom)
```

```
   ↓
2. Submit → INSERT rooms
   ↓
3. Redirect ke /dashboard-admin/manage-rooms
```

---

### Edit Kamar

```
/dashboard-admin/manage-rooms/{kostId}/manage/{roomId}
   ↓
1. Edit data kamar
2. Toggle is_available (dengan konfirmasi jika dari terisi → tersedia)
   ↓
3. If changing to "tersedia":
   ⚠️ CONFIRMATION MODAL
   ↓
4. Submit → Server Action:
   - Hapus booking_requests
   - Hapus payments
   - Hapus penyewa
   - Update room data
   ↓
5. Redirect ke /dashboard-admin/manage-rooms
```

---

### Hapus Kamar

```
Click "Hapus Kamar Permanen"
   ↓
1. Confirmation Modal:
   - User ketik "HAPUS" (case sensitive)
   ↓
2. Submit → Server Action (cascade delete):
   - Delete booking_requests
   - Delete bookings
   - Delete payments
   - Delete penyewa
   - Delete room
   ↓
3. Success Modal
   ↓
4. Redirect ke /dashboard-admin/manage-rooms
```

---

## 📝 Booking Flow (User Side)

### User Browse & Book Room

```
/dashboard-user
   ↓
1. User browse RoomList
   - Filter by kost (jika owner punya >1 kost)
```

```
   ↓
2. Click "Sewa Kamar Ini"
   ↓
3. BookingModal opens:
   - Input durasi sewa (bulan)
   - Review harga & fasilitas
   ↓
4. Submit → Server Action createBookingRequest:
   - Calculate due_date (current_date + duration months)
   - INSERT booking_requests dengan status 'pending'
   - Set booking_expires_at (10 menit dari sekarang)
   ↓
5. Modal close → Notifikasi muncul di bell icon
```

---

### Admin Process Booking

```
/dashboard-admin/booking-request
   ↓
Section: "Permintaan Sewa Aktif"
   ↓
Admin lihat booking dengan status 'pending' atau 'process'
   ↓
Actions:

A. PROSES PESANAN (pending → process)
   - Update status: 'pending' → 'process'
   - User dapat notifikasi untuk bayar

B. TOLAK (pending → rejected)
   - Update status: 'rejected'
   - Update payment jadi 'rejected'
   - Delete booking_request

C. SETUJUI PEMBAYARAN (process → approved)
   1. Validasi room is_available
   2. Update payment: 'pending' → 'approved'
   3. Update booking status: 'approved'
   4. Hapus penyewa lama dari room (jika ada)
   5. INSERT penyewa baru
   6. Update room: is_available = FALSE
   7. Delete booking_request yang approved
   8. Delete booking_requests lain untuk room yang sama
```

---

## 💳 Payment Flow

### User Upload Payment Proof

```
User dapat notifikasi (booking status 'process')
   ↓
Click Bell Icon → NotificationModal
   ↓
```

```
Click "Bayar Sekarang" → PaymentModal
   ↓
STEP 1: Pilih Metode Pembayaran
   - Rekening Bank
   - QRIS
   ↓
STEP 2: Lihat Detail Payment
   - Rekening: Bank name + account number
   - QRIS: Display QR image
   ↓
STEP 3: Upload Bukti Pembayaran
   - Select file (jpg/jpeg)
   - Upload ke Supabase Storage (bucket: payment_file)
   ↓
STEP 4: Success/Error
   - Success: "Bukti Bayar Berhasil Diunggah"
   - Error: "Cek ekstensi file" (coba lagi)
   ↓
Server Action createPayment:
   - Get booking data
   - Get room.price
   - Upload file to storage
   - INSERT payments with status 'pending'
```

---

### Admin Review Payment

```
/dashboard-admin/payment-history
   ↓
Admin lihat payment dengan status 'pending'
   ↓
Click "Lihat" bukti pembayaran
   ↓
Admin review → Lakukan approve di /dashboard-admin/booking-request
```

---

## 👤 Tenant Dashboard Flow

```
User login yang sudah jadi penyewa
   ↓
Auto-redirect ke /dashboard-penyewa
   ↓
Display:
   - Informasi kost & kamar
   - Lokasi (alamat)
   - Biaya per bulan
   - Jatuh tempo pembayaran
   - Status pembayaran:
     * OVERDUE (merah) - terlambat X hari
     * Near Due (kuning) - 7 hari lagi
     * PAID (hijau) - normal
   ↓
Alert notification jika:
```

```
   - Overdue: "Segera lakukan pembayaran"
   - Near due: "Jatuh tempo sudah dekat"
```

---

## 🔔 Real-time Notification System

```
Dashboard User
   ↓
Supabase Realtime Channel: "booking_requests-changes"
   ↓
Listen to postgres_changes:
   - Event: * (all events)
   - Table: booking_requests
   ↓
On change detected:
   - Re-fetch booking data
   - Update notification count
   - Update booking status
   ↓
Bell icon badge shows pending bookings count
```

---

## ⏱️ Expired Booking System

**Automatic Expiry**:

```
Booking created with status 'pending'
   ↓
Set booking_expires_at = NOW() + 10 minutes
   ↓
Cron Job (/api/check-expired-bookings):
   - Run every X minutes (setup di Vercel Cron atau external)
   - Find bookings where:
     * status = 'pending'
     * booking_expires_at < NOW()
     * paid_at IS NULL
   ↓
Actions:
   - Update status: 'pending' → 'expired'
   - Set room: is_available = TRUE
   ↓
User sees expired notification or booking disappears
```

**Manual Countdown** (Client-side):

```
CountdownTimer component
   ↓
Calculate: booking_expires_at - NOW()
   ↓
Display: MM:SS
   ↓
Colors:
   - < 3 minutes: Red (urgent)
```

```
   - >= 3 minutes: Yellow
   ↓
On 00:00: trigger onExpire callback
```

---

# 📄 Pages & Components

## Landing & Auth Pages

### `/` - Landing Page

- Role selection: User atau Admin
- Redirect ke `/login/user` atau `/login/admin`

### `/login/admin` - Admin Login

- Google OAuth button
- Error handling untuk multiple scenarios
- Redirect ke `/auth/callback/admin`

### `/login/user` - User Login

- Google OAuth button
- Simple flow tanpa error handling
- Redirect ke `/auth/callback/user`

### `/referral-input` - Referral Input

- Form input kode referral
- Client-side uppercase transformation
- Validation: tidak boleh kosong
- Error modal dengan retry
- POST ke `/api/validate-referral`

---

## Admin Dashboard

### `/dashboard-admin` - Summary Dashboard

**Layout**: Sidebar + Header + Content

**Features**:

- **Statistics Cards**:
  - Total Properti
  - Total Kamar (tersedia + terisi)
  - Pemesanan (total + pending)
  - Pendapatan (total + pending payments)

- **Recent Activity**:
  - 5 booking terbaru
  - 5 payment terbaru
  - Scrollable list dengan custom scrollbar
- **Quick Actions**:
  - Tambah Kamar
  - Kelola Kamar
  - Manajemen Kost
  - History Pembayaran
- **Summary Cards** (bottom):
  - Kamar Tersedia (% dari total)
  - Pemesanan Disetujui (% dari total)
  - Pendapatan Bulan Ini (% growth)

**Breadcrumbs**: Dynamic dengan UUID resolution ke nama kost/kamar

---

### `/dashboard-admin/manage-rooms` - Manage Rooms

**Flow Logic**:

1. **No Properties**:
   - Empty state
   - Button "Tambah Properti"
2. **Multiple Properties (>1) & No kost_id**:
   - Grid cards untuk pilih properti
   - Card "Tambah Properti Baru" (dashed border)
3. **Single Property OR kost_id selected**:
   - Display property name & address
   - Statistics: Total, Tersedia, Terisi
   - Room cards grid (responsive 1-2-3 columns)
   - Button "Tambah Kamar Baru"

**Room Card**:

- Header dengan status badge (emerald/amber)
- Harga bulanan
- Fasilitas dengan icons
- Button "Kelola Kamar"
- Hover effects

---

### `/dashboard-admin/booking-request` - Booking Management

**2 Sections**:

1. **Permintaan Sewa Aktif**:
    - Status: ‘pending’ atau ‘process’
    - Filter: belum ada payment approved
    - Actions:
        - Pending: “Proses Pesanan” & “Tolak”
        - Process: “Setujui Pembayaran”
2. **Penghuni Aktif**:
    - Data dari tabel `penyewa`
    - Read-only (no action buttons)
    - Display: email, nomor kamar, harga, due date

**Booking Card**:

- User email
- Nomor kamar & harga
- Status badge dengan icon
- Batas pembayaran / start date

---

### `/dashboard-admin/payment-history` - Payment History

**Features**:

- **Dropdown Filter**: Per properti (jika punya >1 kost)
- **Summary Stats**: Total, Lunas, Pending, Terlambat
- **Table Columns**:
    - Properti (jika filter = ‘all’)
    - Penyewa (email)
    - Kamar
    - Jumlah (Rupiah)
    - Jatuh Tempo
    - Status (badge)
    - Bukti (link ke file)

**Empty States**:

- No properties
- No payments untuk selected kost

---

### `/dashboard-admin/payment-method` - Payment Methods

**Layout**: 2 Columns

**Left Column**: Add Payment Method Form

- Type selection: Rekening atau QRIS
- Conditional fields:
    - Rekening: Bank name + Account number
    - QRIS: File upload (PNG/JPG/SVG, max 5MB)
- Upload ke Supabase Storage bucket `payment_methods`

- Server Action: `addPaymentMethod`

**Right Column**: List Payment Methods

- Card per method:
  - Rekening: Bank + Account number
  - QRIS: Link "Lihat Gambar QRIS"
- Empty state jika belum ada

---

## User Dashboard

### `/dashboard-user` - Browse Rooms

**Initial Load Logic**:

1. Check authentication
2. Check if user is active tenant → redirect `/dashboard-penyewa`
3. Fetch kosts via `/api/get-kosts`
4. If not verified → redirect `/referral-input`

**Flow Based on Kosts**:

**A. Owner punya >1 kost & belum pilih**:

- Display kost selection cards
- Click card → set selectedKost → fetch rooms

**B. Owner punya 1 kost OR sudah pilih**:

- Display main dashboard
- Components:
  - Header dengan notification bell
  - Info banner (jika >1 kost): show selected kost + "Ganti Kost" button
  - RoomList: grid room cards

**Real-time Features**:

- Supabase channel untuk `booking_requests` changes
- Auto re-fetch data on changes
- Update notification count

**Modals**:

- BookingModal: triggered by room selection
- NotificationModal: triggered by bell click
- PaymentModal: triggered from notification

---

### `/dashboard-penyewa` - Tenant Dashboard

**Features**:

- Auto-redirect jika user ada di tabel `penyewa`

- Menggunakan Supabase Admin Client (bypass RLS)

**Display**:

**Status Card** (dynamic color):

- **Green**: Status Aktif, ≥8 hari tersisa
- **Yellow**: Segera Jatuh Tempo, ≤7 hari
- **Red**: OVERDUE, sudah lewat jatuh tempo

**Info Cards**:

1. **Informasi Kost**:
    - Nama kost
    - Nomor kamar
    - Fasilitas
2. **Lokasi**:
    - Alamat lengkap
3. **Informasi Pembayaran**:
    - Biaya per bulan (Rupiah)
    - Jatuh tempo (formatted date)
    - Sisa waktu (hari)

**Alert Messages**:

- Overdue: “Harap segera lakukan pembayaran…”
- Near due: “Jatuh tempo sudah dekat…”

**Empty State**: Jika data room/kost tidak ditemukan

---

## Components Deep Dive

### Admin Components

#### DeleteRoomButton.tsx

- Client component dengan Portal rendering
- Confirmation modal dengan input “HAPUS” (case sensitive)
- Success modal setelah delete
- Props: roomId, roomNumber
- Flow: confirmation → server action → success → redirect

#### RoomAvailabilityCheckbox.tsx

- Custom checkbox dengan conditional confirmation
- Asymmetric logic:
    - Unchecked → Checked: Show confirmation modal
    - Checked → Unchecked: Direct change
- Hidden input untuk form submission

- Warning: “data tidak dapat dikembalikan”

**SubmitButton.tsx** (booking-request)

- Reusable submit button dengan loading state
- 3 variants: primary (emerald), danger (red), blue
- Props: buttonText, action, variant, className
- useFormStatus hook untuk pending state

**PaymentMethodForm.tsx**

- Multi-step form dengan type selection
- Conditional fields based on type
- File preview untuk QRIS upload
- Success/error message display
- useActionState untuk server action

**AddPropertyForm.tsx**

- Simple form: nama + alamat
- Client-side validation
- POST ke `/api/kosts`
- Redirect ke manage-rooms dengan new kost_id

**RoomForm.tsx** (add room)

- Predefined facilities dengan icons (7 items)
- Custom facility input dengan Enter key support
- Selected facilities sebagai removable chips
- Price preview dengan toLocaleString
- Back button (router.back)

---

**User Components**

**Header.tsx**

- Title “Cari Kamar Kos”
- Bell icon dengan notification badge
- Badge counter: max 9, show “9+” jika >9
- Click handler untuk open NotificationModal

**RoomList.tsx**

- Grid responsive (1-2-3 columns)
- Empty state dengan icon dan message
- Map rooms ke RoomCard components
- Props: rooms, onSelect

**RoomCard.tsx**

- Room info: number, price, facilities
- Status badge (tersedia/terisi)
- Facilities sebagai chips (comma-separated)
- Button states:

    - Available: “Sewa Kamar Ini” (emerald, clickable)
    - Occupied: “Kamar Sudah Terisi” (gray, disabled)
- Hover effects dengan scale

**BookingModal.tsx**

- Backdrop blur dengan glass effect
- Display room info (number, price, facilities)
- BookingForm component
- Close button (X)
- Terms & conditions text

**BookingForm.tsx**

- Duration input (1-36 months)
- Hidden inputs: room_id, kost_id, duration
- useActionState untuk server action
- SubmitForm component
- Success callback untuk close modal

**SubmitForm.tsx**

- Submit button dengan loading state
- NotificationModal integration
- useFormStatus hook
- Success/Error notification dengan redirect
- Props: formState, onSuccess

**NotificationModal.tsx**

- Headless UI Dialog
- Message: “Booking sedang diproses”
- 2 buttons: “Nanti” dan “Bayar Sekarang”
- Callback: onPayNow(bookingId)

**PaymentModal.tsx**

- Multi-step wizard (4 steps)
- Progress indicator dengan dots
- Step logic:
    1. Pilih metode (rekening/qris)
    2. Display payment details
    3. Upload bukti pembayaran
    4. Success/Error state
- File upload: accept jpg/jpeg only
- Navigation: “Sebelumnya” & “Lanjut” buttons
- Server action: createPayment

**CountdownTimer.tsx**

- Real-time countdown MM:SS format
- Calculate: expiresAt - now
- Update setiap 1 detik
- Color states:
    - <3 minutes: Red (urgent)

    - ≥3 minutes: Yellow
- On expire: trigger callback
- Format: padStart(2, '0')

---

# 🎨 Design System

## Color Palette

**Primary Colors**:

- **Emerald**: `emerald-50` to `emerald-900` (main brand)
- **Teal**: `teal-50` to `teal-900` (accent)
- **Blue**: `blue-50` to `blue-900` (info, user theme)

**Status Colors**:

- **Success/Available**: Green/Emerald
- **Warning/Near Due**: Yellow/Amber
- **Error/Overdue**: Red
- **Info/Processing**: Blue
- **Neutral/Occupied**: Gray

## Typography

**Headings**:

- H1: `text-3xl` or `text-4xl` + `font-bold`
- H2: `text-2xl` + `font-bold` or `font-semibold`
- H3: `text-xl` + `font-semibold`

**Body**:

- Default: `text-base` + `text-gray-600`
- Small: `text-sm` + `text-gray-500`
- Tiny: `text-xs` + `text-gray-400`

## Spacing

**Consistent Padding**:

- Cards: `p-6` or `p-8`
- Sections: `py-8` or `py-12`
- Buttons: `px-4 py-2` or `px-6 py-3`

**Gaps**:

- Grid: `gap-4` or `gap-6`
- Flex: `space-x-2` or `space-x-4`

### Components

**Cards**:

```
.card {
  @apply bg-white rounded-xl shadow-sm border border-gray-200;
  @apply hover:shadow-md transition-shadow;
}
```

**Buttons**:

```
.btn-primary {
  @apply bg-gradient-to-r from-emerald-600 to-teal-600;
  @apply text-white font-semibold px-6 py-3 rounded-xl;
  @apply hover:from-emerald-700 hover:to-teal-700;
  @apply transition-all duration-200 shadow-lg;
}

.btn-secondary {
  @apply bg-white border-2 border-gray-300;
  @apply text-gray-700 font-medium px-6 py-3 rounded-xl;
  @apply hover:bg-gray-50 transition-colors;
}
```

**Badges**:

```
.badge {
  @apply inline-flex items-center space-x-1;
  @apply px-3 py-1 rounded-full text-xs font-medium;
}

.badge-success {
  @apply bg-emerald-100 text-emerald-800 border border-emerald-200;
}

.badge-warning {
  @apply bg-yellow-100 text-yellow-800 border border-yellow-200;
}
```

---

## 🔒 Security & Best Practices

### Row Level Security (RLS)

**Profiles**:

```
-- Users can read their own profile
CREATE POLICY "Users can read own profile"
ON profiles FOR SELECT
USING (auth.uid() = id);
```

```sql
-- Users can update their own profile
CREATE POLICY "Users can update own profile"
ON profiles FOR UPDATE
USING (auth.uid() = id);
```

**Kosts**:

```sql
-- Owners can read their own kosts
CREATE POLICY "Owners can read own kosts"
ON kosts FOR SELECT
USING (auth.uid() = owner_id);

-- Owners can insert kosts
CREATE POLICY "Owners can insert kosts"
ON kosts FOR INSERT
WITH CHECK (auth.uid() = owner_id);
```

**Rooms**:

```sql
-- Anyone can read available rooms
CREATE POLICY "Anyone can read available rooms"
ON rooms FOR SELECT
USING (is_available = true);

-- Owners can manage their rooms
CREATE POLICY "Owners can manage rooms"
ON rooms FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM kosts
    WHERE kosts.id = rooms.kost_id
    AND kosts.owner_id = auth.uid()
  )
);
```

## Service Role Client Usage

**When to use Supabase Admin Client**:

- Bypass RLS untuk operasi admin
- Cascade deletes yang kompleks
- Cross-user operations (admin managing tenants)

**Files menggunakan Admin Client**:

- `/api/validate-referral/route.ts`
- `/auth/callback/admin/route.ts`
- `/dashboard-penyewa/page.tsx`
- `/dashboard-admin/manage-rooms/[kostId]/manage/[roomId]/page.tsx`
- `/dashboard-admin/manage-rooms/actions.ts`

### Environment Variables Security

**Never expose**:

- `SUPABASE_SERVICE_ROLE_KEY` (backend only)
- `CRON_SECRET`

**Safe to expose**:

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- `NEXT_PUBLIC_SITE_URL`

### Input Validation

**Server-side**:

- Always validate form data di server actions
- Sanitize user inputs
- Check authorization sebelum CRUD operations

**Client-side**:

- HTML5 validation attributes
- Required fields
- Type validation (number, email, etc.)
- Custom validation rules

---

## 🧪 Testing

### Manual Testing Checklist

**Authentication**:

- ✅ Admin login dengan Google
- ✅ User login dengan Google
- ✅ Referral code validation
- ✅ Role-based redirects
- ✅ Logout functionality

**Admin Features**:

- ✅ Create/Read/Update/Delete kost
- ✅ Create/Read/Update/Delete room
- ✅ Process booking (pending → process → approved)
- ✅ Reject booking
- ✅ Clear tenant from room
- ✅ Upload QRIS image
- ✅ Add payment method (rekening)
- ✅ View payment history

**User Features**:

- ✅ Browse available rooms
- ✅ Submit booking request
- ✅ Upload payment proof
- ✅ Real-time notification updates
- ✅ View tenant dashboard (after approved)
- ✅ Countdown timer functionality

**Edge Cases**:

- ✅ Expired bookings cleanup
- ✅ Multiple simultaneous bookings untuk same room
- ✅ Changing room availability dengan tenants
- ✅ Deleting room dengan active bookings
- ✅ Owner dengan multiple kosts
- ✅ User tanpa referral verification

---

# 🐛 Troubleshooting

## Common Issues

**Issue**: "User not authenticated" errors

- **Solution**: Check Supabase Auth settings, verify JWT tokens

**Issue**: RLS policy errors

- **Solution**: Use Supabase Admin Client untuk admin operations

**Issue**: File upload gagal

- **Solution**:
  - Verify bucket exists dan public
  - Check file size limits
  - Verify file type (jpg/jpeg/png)

**Issue**: Real-time updates tidak working

- **Solution**:
  - Enable Realtime di Supabase dashboard
  - Check channel subscription
  - Verify table replication settings

**Issue**: Cascade delete errors

- **Solution**: Delete dalam urutan yang benar (lihat Cascade Delete Order)

**Issue**: Booking expired tidak auto-update

- **Solution**:
  - Setup cron job properly

- Check CRON_SECRET environment variable
- Verify cron schedule

---

# 📚 Additional Resources

## Documentation

- Next.js 14 Docs
- Supabase Docs
- Tailwind CSS
- Lucide Icons

## Learning Resources

- Next.js Server Actions
- Supabase Auth
- Supabase Realtime
- Supabase Storage

---

**Built by Rifqi Chusaini with ❤️ using Next.js, Supabase, and Tailwind CSS**

Version: 1.0.0
Last Updated: 2025